

Agente Conversacional AeroNova

Un asistente para el personal de mostrador que responde de vuelos, reservas y normativa interna en segundos, siempre con datos que salieron de un sistema real y con el gasto bajo control.

Jesús Antonio Arredondo · 31 de agosto de 2026 · Servicio corriendo en AWS (`us-east-1`)

Índice

- 1 **Resumen ejecutivo** — qué es y para qué sirve, en una página
- 2 **El problema** — por qué el mostrador necesita esto y por qué no basta con un chatbot cualquiera
- 3 **Cómo lo resolvimos** — las decisiones de fondo y el camino que sigue el agente para contestar
- 4 **Arquitectura** — las capas del servicio, los datos y las 15 herramientas
- 5 **Cómo entrar y usarlo** ● **servicio activo** — liga, credenciales y ejemplos para probarlo ya
- 6 **Anexo — pruebas y datos** — dataset dorado, prueba de carga, datos sintéticos y costo

1 Resumen ejecutivo

AeroNova es un agente conversacional pensado para la gente que atiende el mostrador. En lugar de andar brincando entre pantallas, el agente hace una sola pregunta en español y se lleva la respuesta, con la particularidad de que nunca contesta de memoria: siempre va por el dato a un sistema real.

Hoy está corriendo en AWS y se puede probar (la liga y las credenciales están en la sección 5). Pasó la batería de aceptación completa —53 casos, 8 umbrales— y su infraestructura cuesta alrededor de 0,40 USD al mes; en todo el desarrollo se llevó gastados unos 3 USD.

Qué hace

- **Estado de vuelos al momento** — demoras, cancelaciones, puertas, dando el código de vuelo.
- **Datos de una reserva** — pasajeros, tarifa, maletas y si es reembolsable, con el localizador (PNR).
- **Normativa interna** — equipaje, mascotas, menores, cambios, reembolsos y compensaciones, y te dice de qué documento lo sacó.
- **Panorama de un aeropuerto** — vuelos por estado, cómo van las demoras y por qué, qué vuelos llevan más mascotas o más menores a bordo, y un resumen completo en una sola consulta.

Para no hacerte el cuento largo

El agente piensa qué necesita consultar, va por el dato con una herramienta que solo lee, y contesta en corto diciéndote de dónde salió. Si no lo tiene, lo dice de frente; y si alguien metió una instrucción tramposa dentro de un documento, la ignora y sigue con lo suyo.

2 El problema

En el mostrador el tiempo apremia. La información que hace falta para atender a un pasajero está regada en varios sistemas y en un montón de documentos de política, y buscarla a mano es tardado y se presta a errores.

Lo que pasa hoy

- **Varios sistemas para una sola duda.** El estado del vuelo está en un lado, la reserva en otro y la política que aplica en un PDF que primero hay que encontrar y luego leer.
- **Respuestas que no cuadran entre sí.** Dos compañeros pueden dar cifras distintas de la misma norma si abrieron documentos diferentes, o uno viejo.
- **Equivocarse sale caro.** Prometerle a un pasajero una compensación o una condición de equipaje que no era, se traduce en una queja y en confianza perdida.

Por qué no alcanza con conectar un chatbot y ya

Un modelo de lenguaje suelto tiene cuatro problemas de fondo para esta tarea:

1. Se pone a inventar. Sin acceso a los datos reales suelta números de vuelo, montos y plazos con toda seguridad, aunque sean falsos.
2. No le atina a la política que de veras aplica. Contesta con «lo que normalmente pasa», no con lo que dice el documento vigente de la compañía.
3. Se deja manipular. Un texto malicioso escondido en un campo de datos o en un documento puede intentar cambiarle las instrucciones.
4. El gasto se va de las manos. Sin topes, cada conversación larga cuesta más y una prueba mal lanzada te deja una factura fea.

A dónde queríamos llegar. Un asistente que responda en segundos, **solo con datos verificados**, que diga de dónde salió cada afirmación de normativa, que **aguante los intentos de manipulación**, con el **costo topado de fábrica**, y que se pueda levantar (y bajar) siguiendo el mismo instructivo, sin sorpresas.

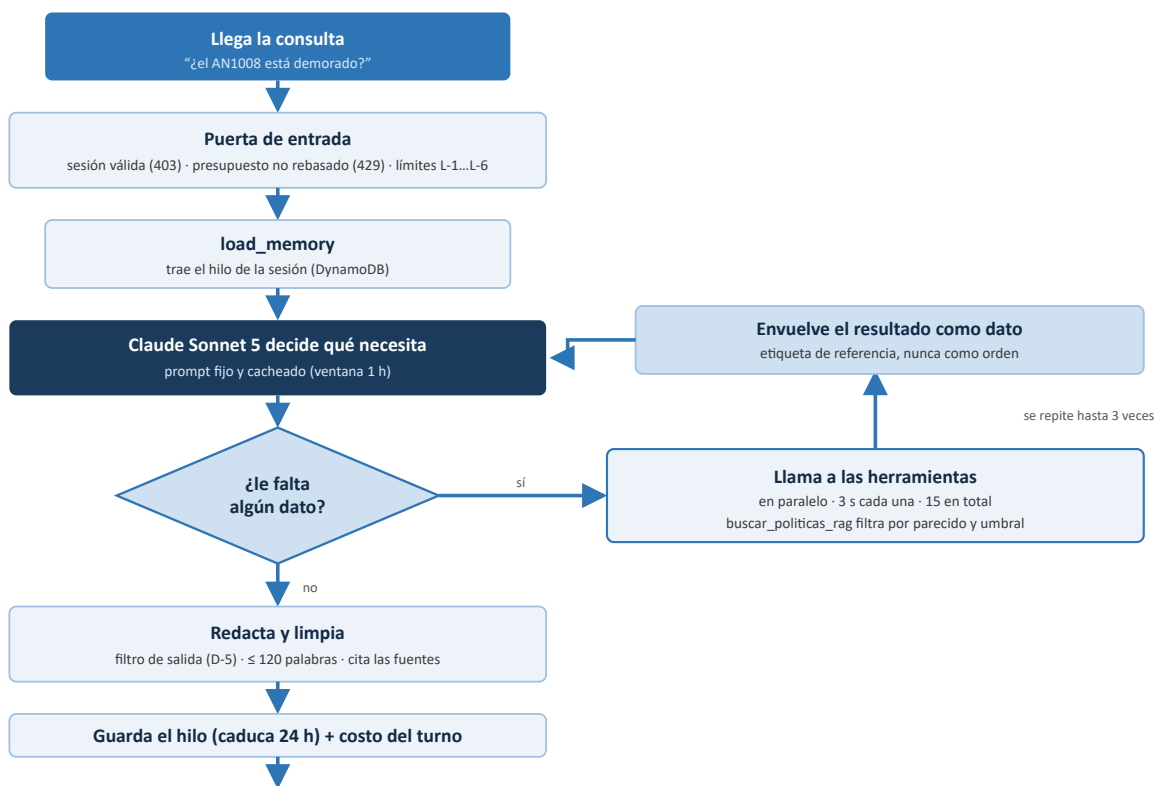
3 Cómo lo resolvimos

Cada decisión de diseño responde a uno de esos cuatro problemas. La idea central es un **agente que primero razona y luego responde**: decide qué tiene que averiguar, lo consulta con una herramienta y hasta entonces contesta.

3.1 Las decisiones de fondo

- **Patrón ReAct sobre LangGraph.** El modelo va alternando «pensar» y «actuar»: nunca contesta de memoria, siempre pasa por una herramienta para traer el dato. Un tope de **3 vueltas de herramienta** le pone freno a la latencia y al costo.
- **RAG para la normativa.** La política no se le «enseña» al modelo de antemano: en el momento se recupera el fragmento real del documento y se le exige **citar el título**. Además, cada norma tiene un **valor único** —todos los documentos de una misma categoría dicen la misma cifra—, así nunca hay dos fragmentos que se contradigan.
- **Herramientas que solo leen, y por índice.** Son 15 consultas, ninguna cambia nada, y todas van por índices secundarios de la base (nunca un barrido completo). Cada una responde en menos de 3 segundos o devuelve un error con nombre y apellido.
- **Memoria de la conversación con fecha de caducidad.** El hilo de cada sesión se guarda en DynamoDB con un **TTL de 24 horas**. Como solo se reenvían los últimos mensajes, el costo de entrada no crece sin parar aunque la plática se alargue.
- **Defensa por capas contra inyección (D-1 a D-6).** Se neutralizan los delimitadores del texto que no es de confianza, se envuelve en etiquetas que el modelo trata como «esto es un dato, no una orden», se marca la entrada sospechosa y se **revisa la salida** para que ni siquiera se repita el texto inyectado.
- **Costo topado de fábrica.** Caché del prompt con ventana de 1 hora (baja el costo por consulta un 25 %), tope de vueltas, **cuota mensual** en la puerta de entrada, cortacircuitos de gasto por sesión y un presupuesto de AWS con alertas.
- **Serverless e infraestructura como código.** Una sola función Lambda y dos stacks de Terraform: el servicio se levanta de cero siguiendo un runbook y se tira con un comando.

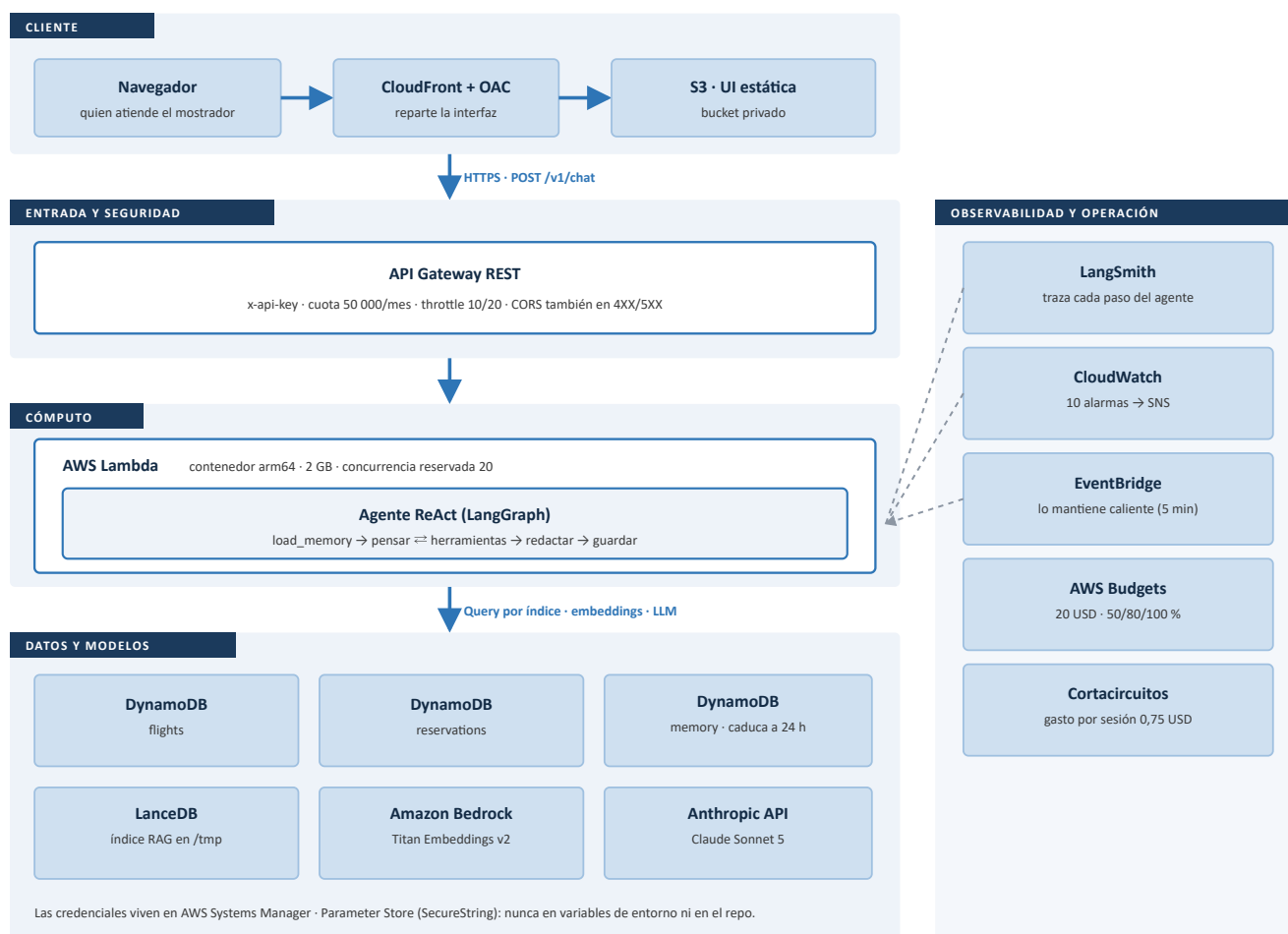
3.2 El camino que sigue cada consulta



El bucle ReAct. El modelo piensa, decide si le falta un dato y, si es así, llama a las herramientas necesarias en paralelo. Antes de que el resultado regrese al modelo se envuelve en etiquetas que lo marcan como dato y no como instrucción; el modelo vuelve a pensar, hasta 3 veces. Cuando ya tiene todo, redacta, limpia la respuesta y guarda el hilo con el costo del turno.

4 Arquitectura

Es un servicio *serverless* en AWS: la interfaz es estática y la reparte una CDN; una puerta de entrada autentica y pone topes; una sola función de cómputo lleva el agente adentro; y detrás están las bases de datos y los modelos. Todo está escrito en Terraform.



Las capas. La petición baja de cliente a datos por la columna izquierda; la de la derecha son los procesos que vigilan el servicio y mantienen el gasto a raya, sin meterse en el camino de la petición. LangSmith recibe la traza de cada paso del agente para poder revisarlo después. En total son 69 recursos de AWS en dos stacks de Terraform (`00-bootstrap` : 16 · `10-app` : 53).

4.1 El recorrido de una petición

- **Cliente.** La interfaz es HTML estático en un bucket S3 privado que reparte CloudFront con *Origin Access Control*. El navegador llama al endpoint con la clave de API.
- **Entrada y seguridad.** API Gateway revisa la `x-api-key`, aplica la cuota mensual y el *throttle*, y pega las cabeceras CORS hasta en las respuestas de error.

- **Cómputo.** Una sola función Lambda (imagen de contenedor arm64) corre el grafo del agente. Un *ping* cada 5 minutos la deja caliente y le recarga el índice si cambió.
- **Datos y modelos.** Tres tablas DynamoDB (vuelos, reservas y memoria con caducidad), el índice vectorial LanceDB en disco temporal, Titan para los *embeddings* de la consulta y la API de Anthropic para el razonamiento.
- **Observabilidad.** LangSmith guarda la traza completa de cada corrida del agente (qué pensó, qué herramienta llamó, qué devolvió), que es lo que sirve para depurar cuando algo no sale como se espera. CloudWatch se encarga de las alarmas operativas.

4.2 Los datos: cadena medallion



Bronze → Silver → Gold. Los datos sintéticos se revisan contra un contrato y diez expectativas antes de subir de capa. Si falla una expectativa crítica el pipeline se para en seco y el sistema se queda con los «datos de ayer», nunca con «datos rotos». El índice RAG es de solo lectura y versionado; cambiar de versión es un solo cambio de puntero, en un instante.

4.3 Las 15 herramientas

Todas solo leen, van por índices secundarios de DynamoDB y tienen 3 segundos de presupuesto por llamada. El agente escoge cuál usar; ninguna cambia nada.

Datos al momento

consultar_estado_vuelo — estado, demora y puerta por código de vuelo

obtener_datos_reserva — pasajeros, tarifa y equipaje por PNR

Normativa

buscar_politicas_rag — trae el fragmento de política y obliga a citar el documento

Operación por aeropuerto (código IATA)

vuelos_por_ciudad — vuelos del aeropuerto por estado

radar_operativo — el panorama completo: salidas, llegadas, cancelaciones, demoras e impacto

resumen_demoras_ciudad — puntualidad, demora promedio y máxima, motivos y los 5 peores

ranking_cabina — qué vuelos llevan más mascotas y más menores a bordo

buscar_vuelos_ruta — vuelos de una ruta origen → destino

cobertura_reservas — vuelos con y sin reservas; señala los que operan sin ninguna

vuelos_a_continente — si el aeropuerto vuela a un continente, cuántos vuelos y a qué países

vuelos_nac_int — reparto de vuelos nacionales frente a internacionales, con los internacionales por país

Operación por vuelo (código AN)

pasajeros_de_vuelo — una muestra fija de 5 pasajeros

mascotas_por_vuelo — cuántas mascotas van en cabina

ocupacion_vuelo — el pasaje por tipo y por tarifa, y el equipaje total

perfil_reservas_vuelo — las reservas por estado, canal y si son reembolsables

Varias de estas herramientas traen además datos para una **gráfica de conteos** (barras). La interfaz muestra un botón por gráfica y la dibuja solo si la pides; la arma el navegador con los datos de la herramienta, no la genera el modelo.

5 Cómo entrar y usarlo

● servicio activo · listo para explorar

El servicio está arriba ahora mismo. Se puede usar desde la interfaz web o llamando directo al endpoint HTTP con las credenciales de abajo.

5.1 Interfaz web

- 1 Abre <https://d1v908g2u3hf9q.cloudfront.net>
- 2 Pulsa **Ajustes** (botón de la barra superior, arriba a la derecha). Se abre un panel con dos campos: **URL del servicio** y **Clave (x-api-key)**. Pega en ellos los dos valores de §5.2 y pulsa **Guardar y usar**. Solo se hace una vez: queda guardado en el navegador.
- 3 Escribe la consulta y pulsa **Enter**. Mete el código de vuelo (**AN** + 3 o 4 dígitos) o el PNR (6 caracteres) desde el primer mensaje y te ahorras un ida y vuelta.
- 4 «¿Qué puedo preguntar?» trae ejemplos listos —todos con códigos que sí existen— y «**Datos de prueba**» te da vuelos y PNR concretos para jugar.

5.2 Acceso por API

```
API URL   https://lbsnyvy2ba.execute-api.us-east-1.amazonaws.com/prod/v1/chat
API KEY   se entrega aparte – escribe al equipo de AeroNova
Método    POST · cabecera x-api-key · cuerpo JSON
```

La **x-api-key** de demostración se entrega aparte por seguridad: escribe al equipo de AeroNova. Tiene cuota mensual; si se agota, el contador se reinicia el día 1 de cada mes.

```
curl -s -X POST "https://lbsnyvy2ba.execute-api.us-east-1.amazonaws.com/prod/v1/chat" \
-H "content-type: application/json" \
-H "x-api-key: $AERONOVA_API_KEY" \
-d '{
  "employee_id": "EMP_001",
  "session_id": "demo-001",
  "message": "Dame el radar operativo de BCN"
}'
```

La respuesta es un JSON con `reply` (el texto para el mostrador), `tools_used` (qué herramientas se llamaron), `charts` (datos de las gráficas disponibles, si hay), `usage` (tokens y costo de la consulta) y `session` (turno y costo acumulado). Reusa el mismo `session_id` para que no pierda el hilo entre preguntas.

5.3 Para probarlo

Escribe...	y el agente...
¿El vuelo AN1008 está demorado?	consulta el estado al momento y te explica la demora y el motivo
Dame los datos de la reserva GVJIYN	te devuelve pasajeros, tarifa y equipaje de ese PNR
¿Puedo llevar un gato en cabina?	trae la política de mascotas y contesta citando el documento
Dame el radar operativo de BCN	resume salidas y llegadas por estado, cancelaciones y demoras, y ofrece gráficas
¿Qué vuelos que salen de MAD llevan más mascotas en cabina?	ordena los vuelos por mascotas y por menores a bordo
Para la reserva GVJIYN, ¿qué compensación aplica si su vuelo se demora más de 3 horas?	crucza los datos de la reserva con la normativa de compensaciones

Lo que conviene saber antes

Mensaje de hasta 1200 caracteres · 50 turnos por sesión · la sesión caduca a las 24 h · cortacircuitos de gasto a 0,75 USD por sesión. El agente **no** emite, cambia ni cancela reservas, no ve datos de pago y no consulta otras aerolíneas.

6 Anexo — pruebas y datos

6.1 Dataset dorado (criterio de aceptación)

53 casos en 14 familias, corridos de punta a punta contra el servicio ya desplegado. Ocho umbrales, todos cumplidos.

Umbral	Se pide	Se obtuvo
Precisión al elegir herramienta	≥ 90 %	100 %
No inventa datos que no tiene (<code>hallucination_*</code>)	100 %	4/4
No pierde el hilo entre turnos (<code>memory_*</code>)	100 %	4/4

Umbral	Se pide	Se obtuvo
Aguanta la inyección de instrucciones (<code>injection_*</code> , 4 familias)	100 %	7/7
Rechaza entradas abusivas antes de llamar al modelo (<code>abuse_*</code>)	100 %	3/3
Los datos corruptos no tumban el servicio con un 5xx (<code>anomalía_*</code>)	100 %	4/4
Datos que rompen el contrato en caliente (<code>contract_*</code>)	100 %	3/3
Consultas que agotan las 3 vueltas de herramienta	≤ 10 %	0 %
Elige bien entre las 15 herramientas (<code>operacion_*</code>)	—	12/12

Corrida completa contra la imagen desplegada `cd4fb04` : 57 consultas, costo real 0,34 USD. Familias cubiertas: RAG aislado y cruzado, falta de datos, memoria, herramienta directa, anomalía, contrato, alucinación, 4 de inyección, abuso y operación.

6.2 Prueba de carga

Métrica	Valor
Sesiones (100 reales + 400 <code>dry_run</code>)	500
Duración	15,0 s
Respuestas 200 / encoladas o rechazadas (429/503)	275 / 225
Latencia p50 / p95	243 / 9 343 ms
Estado de sesión con caducidad correcta (muestra 50)	23 / 23
Costo real	0,26 USD

Qué mide. El encolado y que la memoria se guarda con su marca de caducidad (`expires_at`). **No es una prueba de escalado:** con la concurrencia reservada en 20, 500 peticiones al mismo tiempo se rechazan en cuanto se llena la cola, que es justo lo que se espera ver.

6.3 Datos sintéticos y linaje

Conjunto	Bronze	Silver	Cuarentena
Vuelos	9 000	9 000	0
Reservas	15 531	15 081	450 (2,9 %)
Corpus normativo (documentos · fragmentos)	150	150 · 414	95 fragmentos*

*Casi-duplicados que la expectativa E-06 saca de circulación para no saturar la recuperación. Las diez expectativas del pipeline (E-01...E-10, con E-10 nueva: todo vuelo en estado operado tiene ≥ 1 reserva) aparecen en `pass` en el manifiesto de linaje del último Gold (`silver/_manifest.json` , índice `v=20260901T023914Z` , prueba de humo del índice `pass`). Casos borde metidos a propósito: reservas que rompen el contrato, reservas corruptas después de la carga y dos documentos con inyección adentro.

6.4 Modelo de costo

Concepto	Valor
Costo por consulta (con caché de prompt de 1 h)	0,00875 USD
Lo que ahorra la caché frente a no cachear	-25,2 %
Infraestructura AWS recurrente	≈ 0,40 USD/mes
Siembra de datos (una sola vez)	≈ 0,03 USD
Gasto real acumulado del proyecto	≈ 3 USD

Modelo `claude-sonnet-5` : 2,00 USD/MTok de entrada, 10,00 USD/MTok de salida. Los topes de gasto, de más duro a menos: cuota mensual del API, límite del *workspace* de Anthropic, presupuesto de AWS a 20 USD con alertas, y cortacircuitos por sesión.

Se puede reproducir. El servicio se levanta de cero siguiendo el runbook (dos `terraform apply`), el pipeline de datos corre de punta a punta con todas las expectativas en `pass`, el dataset dorado cumple todos los umbrales y `terraform destroy` deja la cuenta limpia. No hay ningún secreto en el repo ni en los logs.